📑 جزوه سیستم عامل - فصل ۶: ابزار همگامی (بخش دوم)

(پشتیبانی سخت‌افزاری: دستورات اتمیک و متغیرهای یکپارچه)

۶. پشتیبانی سخت‌افزاری برای همگامی (Hardware Support)

همان‌طور که در بخش قبل دیدیم، راهکارهای نرم‌افزاری خالص (مثل پترسن) در معماری‌های مدرن تضمین شده نیستند و کند اجرا می‌شوند. بنابراین، سیستم عامل از **سخت‌افزار** کمک می‌گیرد.

۶-۱. سدهای حافظه (Memory Barriers)

در پردازنده‌های مدرن، برای افزایش سرعت، ممکن است ترتیب اجرای دستورات تغییر کند (Instruction Reordering).

- **مدل حافظه (Memory Model):** نحوه برخورد پردازنده با ترتیب دستورات.
  - **ترتیب قوی (Strongly Ordered):** تغییرات یک پردازنده بلافاصله به بقیه نشان داده می‌شود.
  - **ترتیب ضعیف (Weakly Ordered):** تغییرات ممکن است با تأخیر دیده شوند.

راه حل: استفاده از دستور memory_barrier().

- این دستور به پردازنده می‌گوید: "تمام دستورات نوشتن/خواندن قبل از من را تمام کن، بعد برو سراغ دستورات بعدی".
- **مثال کاربردی:**

```
// نخ ۱
x = 100;
memory_barrier(); // تضمین می‌کند x حتما ۱۰۰ شده باشد
flag = true;      // قبل از اینکه flag تغییر کند
```

۶-۲. دستورات سخت‌افزاری اتمیک (Atomic Instructions)

سخت‌افزار دستوراتی را ارائه می‌دهد که **اتمیک (Atomic)** هستند.

معنی اتمیک: یعنی "تجزیه‌ناپذیر". این دستورات به‌صورت یک واحد پیوسته اجرا می‌شوند و **وقفه (Interrupt)** وسط آن‌ها رخ نمی‌دهد. هیچ دو پردازنده‌ای نمی‌توانند هم‌زمان این دستورات را روی یک متغیر اجرا کنند.

الف) دستور test_and_set

این دستور دو کار را در یک لحظه انجام می‌دهد:

1. مقدار فعلی یک متغیر را برمی‌گرداند (می‌خواند).
2. مقدار آن متغیر را به TRUE تغییر می‌دهد.

```
} boolean test_and_set(boolean *target)
    boolean rv = *target; // مقدار قبلی را ذخیره کن
    *target = TRUE;       // مقدار جدید را True کن
    return rv;            // مقدار قبلی را برگردان
{
```

✅ **پیاده‌سازی انحصار متقابل (Mutex) با `test_and_set` :**

```
// متغیر مشترک lock با مقدار اولیه false
while (test_and_set(&lock)); // تا وقتی قفل است (True برمی‌گرداند)، همین‌جا درجا بزن

/* --- بخش بحرانی (Critical Section) --- */

lock = false; // قفل را آزاد کن

/* --- بخش باقیمانده --- */
```

تحلیل: اگر lock مقدارش false باشد، test_and_set آن را true می‌کند و خودش false برمی‌گرداند -> حلقه می‌شکند و پردازه وارد می‌شود. بقیه پشت در می‌مانند چون حالا lock مقدارش true است.

ب) دستور CAS (compare_and_swap)

این دستور پیچیده‌تر و قدرتمندتر است. روی ۳ مقدار کار می‌کند:

1. value: آدرس متغیر اصلی.
2. expected: مقداری که انتظار داریم متغیر داشته باشد.
3. new_value: مقدار جدیدی که می‌خواهیم بنویسیم.

منطق: "اگر مقدار فعلی value برابر با expected بود، آن را به new_value تغییر بده. در غیر این صورت دست نزن. در هر حال مقدار اولیه را برگردان."

تعریف شبه‌کد:

```
} int compare_and_swap(int *value, int expected, int new_value)
                                                    ;int temp = *value
                                                if (*value == expected)
                                                ;value = new_value*
                                                          ;return temp
                                                                    {
```

✅ **پیاده‌سازی انحصار متقابل با CAS:**

```
// متغیر lock با مقدار اولیه 0
(while (compare_and_swap(&lock, 0, 1) != 0); // اگر 0 است، 1 کن و وارد شو. اگر 1 است، صبر کن.

/* --- بخش بحرانی --- */

lock = 0; // آزادسازی قفل

/* --- بخش باقیمانده --- */
```

۶-۳. چالش بزرگ: حل مشکل "انتظار محدود" با CAS

(نکته کنکوری و امتحانی) الگوریتم‌های ساده‌ی بالا (بخش ۲-۶) شرط **"انحصار متقابل"** و **"پیشروی"** را دارند، اما **"انتظار محدود" (Bounded Waiting)** را تضمین نمی‌کنند (ممکن است یک پردازه تا ابد در حلقه while بماند و گرسنگی بکشد).

برای حل این مشکل، از آرایه‌ای به نام waiting[] استفاده می‌کنیم تا نوبت را حفظ کنیم.

الگوریتم انحصار متقابل + انتظار محدود (با استفاده از CAS):

```
// متغیرهای مشترک: boolean waiting[n], int lock = 0
} while (true)
    waiting[i] = true; // من منتظرم
    key = 1;

    // حلقه انتظار: تا وقتی منتظرم AND قفل دست کس دیگری است
    while (waiting[i] && key == 1)
        key = compare_and_swap(&lock, 0, 1); // سعی کن قفل را بگیری

    waiting[i] = false; // دیگر منتظر نیستم (یا قفل را گرفتم یا نوبتم شد)

    /* --- بخش بحرانی --- */

    // پیدا کردن نفر بعدی برای دادن نوبت (به صورت چرخشی)
    j = (i + 1) % n
    while ((j != i) && !waiting[j]) // برو جلو تا برسی به اولین کسی که منتظر است
        j = (j + 1) % n

    if (j == i)      // هیچکس منتظر نبود
        lock = 0;    // قفل را کلاً آزاد کن
    else             // نفر j منتظر است
        waiting[j] = false; // مستقیماً به او اجازه بده (بدون تغییر lock)

    /* --- بخش باقیمانده --- */
{
```

تحلیل این الگوریتم (چرا ۲۰ می‌گیرید؟):

1. **انحصار متقابل:** تضمین می‌شود چون فقط کسی که lock را 0 به 1 تبدیل کند یا waiting او false شود وارد می‌شود.
2. **انتظار محدود:** وقتی پردازه‌ای خارج می‌شود، لیست را چک می‌کند و به **اولین نفر بعدی** اجازه می‌دهد. پس هر کس حداکثر n-1 بار صبر می‌کند.

۶-۴. متغیرهای یکپارچه (Atomic Variables)

گاهی اوقات فقط می‌خواهیم یک عدد را زیاد یا کم کنیم و نمی‌خواهیم درگیر قفل و کد پیچیده شویم.

- سیستم عامل نوع داده‌های خاصی مثل AtomicInteger ارائه می‌دهد.
- عملیات روی این متغیرها (مثل increment) اتمیک است و از CAS در سطح پایین استفاده می‌کند.

مثال:

```
increment(&sequence);
// این دستور توسط CPU تضمین می‌شود که وسطش وقفه نیفتد و تداخل پیش نیاید.
```

⚠ **نکته مهم:** متغیرهای اتمیک فقط "مسابقه روی یک متغیر" را حل می‌کنند. اگر منطق برنامه پیچیده باشد (مثل مسئله با فر محدود که شرط `count>0` داریم)، متغیر اتمیک به تنهایی کافی نیست و باید از ابزارهای قوی‌تر (مثل سمافور یا مانیتور که در بخش بعد می‌خوانیم) استفاده کنیم.